COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:	Adrian Walter D. Santos	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 **Searching Techniques**

1. PROCEDURE OUTPUT

Screenshot

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
#include "node.h"
#include "searchAlgo.h"

const int max_size = 50;

int main() {
    //generate random values:
    int dataset[max_size];
    srand(time(0));
    for(int i=0; i<max_size; i++){
        dataset[i] = rand();
    }
    for(int i=0; i < max_size; i++){
        std::cout << dataset[i] << " ";
    }
    std::cout<<std::endl;
    //create a user input to ask what value to find:
    int toFind;
```

```cpp
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node{
public:
    T data;
    Node *next;
};

template <typename T>
Node<T> *new_node(T newData){
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}
```

Node.h file

```
18307 314 4420 24330 7098 21646 31378 32634 4764 8086 25014 27552 31406 11790 32130 28888 22131 30909 8889 5032 20450 6928 29474 21133 23 19458 730 1531 24393 24126 26606 415
 28924 2952 26665 23271 120 3929 14635 30916 377 21705 1723 20655 26920 15282 13151 29930 4938 14611
```

Observation
The initial code was useful to create the data that our algorithm will search through, the code was able to generate randomized values using srand from the stdlib library. The second picture shows the node.h that will be used on the linked lists.

Table 6-1. Data Generated and Observations.

Code

```cpp
template <typename T>
void linearSearchArr(T data[], int arrSize, T dataFind){
    //Step 1: I := 0
    int i = 0;

    //Repeat while I <= n
    while( i < arrSize){
        if(dataFind == data[i]){
            std::cout<<"the search is success\n";
            return;
        }
        //increment  i to move to the next element
        i++;
    }
    std::cout<< "the element is not found \n";

    return;

}
```

```cpp
int toFind;
std::cout << " \n\n============================\n";
std::cout << "Linear Search Array:\n";
std::cout << "what value you want to search? ";
std::cin >> toFind;

linearSearchArr(dataset, max_size, toFind);
```

Output

```
20400 17752 20180 10482 21779 1712 24458 7388 6770 6526 11168 9049 24070 998 7742 16955 30821 15861 2509 4764 11606 3879 8040 10877 32484 25900 31373 18861 14001 2027 5401 19
949 6541 20168 14195 12881 31047 15138 14599 7657 6339 7200 9462 17825 5709 8110 5899 9574 20525 29890


==========================
Linear Search Array:
what value you want to search? 53
the element is not found
```

Observations
In a linear search for arrays, the code checks the data one by one by incrementing its index to manually check every element in the array. Then if the data inputted by the user matches the current index then it will be deemed as a success, while if the program goes through the entire array without matching a single element then it will fail.

Table 6-2a. Linear Search for Arrays

Code

```
std::cout << "\nString\n";
std::cout << "JIMLORD";

Node<char>* name1 = new_node('J');
Node<char>* name2 = new_node('I');
Node<char>* name3 = new_node('M');
Node<char>* name4 = new_node('L');
Node<char>* name5 = new_node('O');
Node<char>* name6 = new_node('R');
Node<char>* name7 = new_node('D');

name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;
name6->next = name7;

std::cout << "\nLinear Search Linked list\n";
std::cout << "Linear Search for Linked List result (D): ";
linearSearchLL(name1, 'D');
std::cout << "Linear Search for Linked List result (X): ";
linearSearchLL(name1, 'X');
```

Main code

```cpp
template <typename T>
void linearSearchLL(Node<T>* node, T dataFind){
    //repeat the loop unting the current node is not null
    while(node != nullptr){

        // if item = current node's data then display searching is successful
        if(dataFind == node->data){
            std::cout<< "the search was a success\n";
            return;
        }

        //if not, move to the next node
        node = node->next;
    }

    //the searching reached nullptr, -> search is unsuccessful
    std::cout<<"the search was unsuccessful\n";
}
```

Header File Code

Output

```
String
JIMLORD
Linear Search Linked list
Linear Search for Linked List result (D): the search was a success
Linear Search for Linked List result (X): the search was unsuccessful
```

Observations

This code is for the linked list linear search, the program first assigns the nodes into the string Jimlord then through the code it was able to search through the entire linked list in order to find the data inputted which is D and X.Though unlike the linear search array the node doesn't increment it just traverses by pointing into the next node through node->next

Table 6-2b. Linear Search for Linked List

Code

```cpp
template <typename T>
void binarySearchArr(const T arr[], int n, const T& no) {
    // Step 1: low := 0, up = n - 1
    int low = 0;
    int up = n - 1;

    // Step 2: Repeat while low <= up
    while (low <= up) {
        int mid = (low + up) / 2;

        if (no == arr[mid]) {
            std::cout << "Search element is found!\n";
            return; // exit function
        }
        else if (no < arr[mid]) {
            up = mid - 1;
        }
        else {
            low = mid + 1;
        }
    }

    // Step 3 & 4: Print not found and Exit
    std::cout << "Search element is not found\n";
}
```

Header file

```
std::cout << " \n\n============================";
std::cout << " \nBinary Search\n";

int sortedData[max_size];
for (int i = 0; i < max_size; i++) {
    sortedData[i] = rand() % 100;
}


std::sort(sortedData, sortedData + max_size);

std::cout << "Sorted Array: ";
for (int i = 0; i < max_size; i++) {
    std::cout << sortedData[i] << " ";
}
std::cout << std::endl;

int toFindSorted;
std::cout << "what value you want to search for (binary search)? ";
std::cin >> toFindSorted;

binarySearchArr(sortedData, max_size, toFindSorted);
```

Main code

Output

```
========================
Binary Search
Sorted Array: 0 1 1 5 7 9 9 12 12 14 16 17 18 18 18 20 23 23 24 27 29 29 32 35 36 42 42 43 45 46 47 50 51 53 53 59 61 64 65 69 70 71 77 77 79 79 79 82 88 95
what value you want to search for (binary search)? 3
Search element is not found
```

Observations

The code fills an array with random numbers, sort them in order, and ask the user which number they want to find. Next,it passes the array, its size, and the target number into the binary search function. Finally, the function checks the middle item repeatedly until it finds the number or tells the user it isn't in the list.

Table 6-3a. Binary Search for Arrays

Code

```cpp
template <typename T>
Node<T>* getMiddle(Node<T>* start , Node<T>* last){
    //use two pointers: 1. slow moves one node at a time, fast moves two at a time
    Node<T>* slow = start;
    Node<T>* fast = start;

    //get middle:
    while(fast != last && fast->next != last){
        //move slow (one step)
        slow = slow->next;
        //move sat (two steps)
        fast = fast->next->next;
    }

    //when fast reaches the end
    // slow poitns to the middle node
    return slow;
}
```

Getmiddle header code

```cpp
Node<T>* binarySearchLL(Node<T>* head, T key) {
    // 1. Start node is set to head of the list and the last node is set to NULL.
    Node<T>* start = head;
    Node<T>* last = nullptr;

    do {
        // 2. Middle element is calculated using the two pointers approach discussed above.
        Node<T>* mid = getMiddle(start, last);

        // If list is empty or segment is invalid
        if (mid == nullptr) {
            return nullptr;
        }

        // 3. If the middle element is same as the key to be searched, we return it.
        if (mid->data == key) {
            return mid;
        }

        // 4. Else if middle element is greater than the key to be searched, we have to search is the
        // right side of the singly linked list. So, we set start pointer to the next of middle element.
        else if (mid->data < key) {
            start = mid->next;
        }

        // 5. Else if middle element is less than the key to be searched, we have to search is the left
        // side of the singly linked list. So, we set last pointer to the middle element.
        else {
            last = mid;
        }
    // 6. If the key is found or the entire linked list gets traversed, we break the loop.
    } while (last == nullptr || last != start);
    return nullptr; // Key not found
}
```

Binary Search LL header code

```cpp
Node<char>* n1 = new_node('D');
Node<char>* n2 = new_node('I');
Node<char>* n3 = new_node('J');
Node<char>* n4 = new_node('L');
Node<char>* n5 = new_node('M');
Node<char>* n6 = new_node('O');
Node<char>* n7 = new_node('R');

n1->next = n2;
n2->next = n3;
n3->next = n4;
n4->next = n5;
n5->next = n6;
n6->next = n7;

std::cout << "\nSorted Linked List: D I J L M O R\n";
```

```
Node<char>* midNode = getMiddle(n1, n7);
if (midNode != nullptr) {
    std::cout << "Middle element of full Linked List: " << midNode->data << std::endl;
}

std::cout << "\nBinary Search LL ('D'): ";
Node<char>* resultFound = binarySearchLL(n1, 'D');
if (resultFound != nullptr) {
    std::cout << "Search success\n";
} else {
    std::cout << "Search failed.\n";
}

std::cout << "Binary Search LL ('X'): ";
Node<char>* resultNotFound = binarySearchLL(n1, 'X');
if (resultNotFound != nullptr) {
    std::cout << "Search success\n";
} else {
    std::cout << "Search failed.\n";
}
```

Main code

Output

```
Sorted Linked List: D I J L M O R
Middle element of full Linked List: L

Binary Search LL ('D'): Search success
Binary Search LL ('X'): Search failed.
```

Observations

First, I set up a helper function that uses a fast and slow runner to find the middle node of a linked list. Next, I start searching from the top of the list down to the end to grab that middle spot. Finally, I keep cutting the list in half based on whether the target value is bigger or smaller, returning the matching node when I hit it.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE

```
#include "node.h"
```

```
#ifndef SEARCHALGO_H
#define SEARCHALGO_H

#include <iostream> //no more "void" method data types
#include "node.h"

//ARRAY = SEQUENTIAL SEARCH
template <typename T>
int linearSearchArr(int arrSize,T dataFind ,T list[], int &what, bool
countAll = true){
    int manyHow = 0; what = 0;
    //for loop
    for (int i = 0; i < arrSize; i++){
        what++;
        if (dataFind == list[i]){
            manyHow++;
            if(!countAll){
                return manyHow;
            }
        }
    }
    return manyHow;
}

//BINARY SEARCHING
//GET MIDDLE=-=-=-=-=-=-=-=-=-=
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){
    Node<T>* slow = start;
    Node<T>* fast = start;

    while (fast != last && fast->next != last){
        //slow = 1 step
        slow = slow->next;

        //fast = 2 steps
        fast = fast->next->next;
```

```
    }
    return slow;
}
//=--=--=-=-=-=-=-=-=-=-=-=-=

//LINKED LIST = BINARY SEARCH
template <typename T>
bool binarySearchLL(Node<T>* start, T dataFind){
    //Set last to null, start to head
    Node<T>* last = nullptr;
    Node<T>* current = start;

    //usage of getMiddle
    while (start != last){
        Node<T>* mid = getMiddle(start, last);
        //if middle == dataFind
        if(mid->data == dataFind){
            std::cout<<"Search is successful!"<<std::endl;
            return true;
        //if mid > dataFind
        }else if (mid->data > dataFind){
            last = mid;
        //if mid < dataFind
        }else if(mid->data < dataFind){
            start = mid->next;
        }
    }
    std::cout << "Searching is unsuccessful!" << std::endl;
    return false;
}

//ARRAY = BINARY SEARCH (ITERATIVE)
template <typename T>
bool binarySearchArr(int arrSize, T dataFind, T list[], bool showTimes
= true){
    //define low high and mid
    int low = 0, up = arrSize-1, middle, iterations = 1;
```

```
    //while low is less than middle, high
    while (low <= up){
        //compute for mid
        middle = (int)(low+up)/2;
        if (showTimes){
            std::cout<<"Iteration #"<<iterations<<std::endl;
            std::cout<<"Low = "<<low<<std::endl;
            std::cout<<"Middle = "<<middle<<std::endl;
            std::cout<<"Up = "<<up<<std::endl;
            std::cout<<"Value = "<<list[middle]<<std::endl;
        }
        //if dataFind is middle->data, return
        if (dataFind == list[middle]){
            std::cout<<"Search is successful!\n";
            return true;
            //if dataFind < middle, new up
        }else if(dataFind < list[middle]){
            up = middle - 1;
            //else update low
        }else{
            low = middle +1;
        }
        iterations++;
    }
    std::cout<<"Search is unsuccessful!"<<std::endl;
    return false;
}

//ARRAY = BINARY SEARCH (RECURSIVE)
template <typename T>
bool binarySearchArrRecursive(int low, int up, T dataFind, T list[],
int iterations = 1){
    if (low > up){
        std::cout<<"Searching was unsuccessful!"<<std::endl;
        return false;
    }
```

```
    int middle = (low + up) / 2;

    std::cout<<"Iteration #"<<iterations<<std::endl;
    std::cout<<"Low = "<<low<<std::endl;
    std::cout<<"Middle = "<<middle<<std::endl;
    std::cout<<"Up = "<<up<<std::endl;
    std::cout<<"Value = "<<list[middle]<<std::endl;

    if (dataFind == list[middle]){
        std::cout<<"Search was successful!"<<std::endl;
        return true;
    }else if(dataFind < list[middle]){
        return binarySearchArrRecursive(low, middle - 1, dataFind,
list, iterations + 1 );

    }else{
        return binarySearchArrRecursive(middle + 1, up, dataFind, list,
iterations + 1);
    }
}
#endif
```

Header File

```
#include <iostream>
#include "node.h"
#include "searchAlgo.h"

//PROBLEM 1 & 2 ARRAY
int seqArray[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

int main(){
    //LL IMPLEMENTATION
    Node<int>* seq1 = new_node(15);
    Node<int>* seq2 = new_node(18);
    Node<int>* seq3 = new_node(2);
    Node<int>* seq4 = new_node(19);
```

```
    Node<int>* seq5 = new_node(18);
    Node<int>* seq6 = new_node(0);
    Node<int>* seq7 = new_node(8);
    Node<int>* seq8 = new_node(14);
    Node<int>* seq9 = new_node(19);
    Node<int>* seq10 = new_node(14);

    seq1->next = seq2;
    seq2->next = seq3;
    seq3->next = seq4;
    seq4->next = seq5;
    seq5->next = seq6;
    seq6->next = seq7;
    seq7->next = seq8;
    seq8->next = seq9;
    seq9->next = seq10;

    //==================================
    //PROBLEM #1
    //==================================

    int comparisonArray = 0;
    int similarityArray = linearSearchArr(10, 18, seqArray,
comparisonArray, false);

    int comparisonLL = 0;
    int similarityLL = LinearSearchLL(seq1, 18, comparisonLL, false);

    std::cout << "============================" << std::endl;
    std::cout << "PROBLEM 1: SEQUENTIAL SEARCH" << std::endl;
    std::cout << "Searching for key 18" << std::endl;

    std::cout << "Array comparisons: "<< comparisonArray << std::endl;

    std::cout << "Linked list comparisons: "<< comparisonLL <<
std::endl;
```

```
    std::cout << "Answer: 2 comparisons are needed to find the first
18."<< std::endl;

    //==================================
    //PROBLEM #2
    //==================================

    int countComparisonArray = 0;
    int countComparisonLL = 0;

    //for array
    int similarsArray = linearSearchArr(10, 18, seqArray,
comparisonArray, true);

    //for ll
    int similarsLL = LinearSearchLL(seq1, 18, comparisonLL, true);

    std::cout << "==============================" << std::endl;
    std::cout << "PROBLEM 2: COUNT OCCURRENCES"<< std::endl;

    std::cout << "Array occurrences of 18: "<< similarsArray <<
std::endl;

    std::cout << "Linked list occurrences of 18: "<< similarsLL <<
std::endl;

    std::cout << "Answer: 18 occurs 2 times."<< std::endl;

    //==================================
    //PROBLEM #3
    //==================================

    //array for 3 & 4
    int binaryArray[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    std::cout << "=============================="<< std::endl;

    std::cout << "PROBLEM 3: ITERATIVE BINARY SEARCH"<< std::endl;
```

```
    std::cout << "Searching for key 8"<< std::endl;

    binarySearchArr(10, 8, binaryArray);

    //================================
    //PROBLEM #4
    //================================
    std::cout << "============================="<< std::endl;

    std::cout << "PROBLEM 4: RECURSIVE BINARY SEARCH"<< std::endl;

    std::cout << "Searching for key 8"<< std::endl;

    binarySearchArrRecursive(0, 9, 8, binaryArray);
    std::cout << "============================="<< std::endl;


    return 0;
}
```

Main file

```
#ifndef NODE_H
#define NODE_H
#include <iostream> //we change LinearSearchLL from "void" to "int"...

template <typename T>
class Node{
public:
    T data;
    Node *next;
};

//node creation
template <typename T>
Node <T>* new_node(T newData){
    Node<T> *newNode = new Node<T>;
```

```
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

//EDITED FOR PROBLEM 1 & 2
template <typename T>
int LinearSearchLL(Node<T>* head, T dataFind, int &what, bool countAll
= true){
    int howMany = 0; what = 0;


    while (head != nullptr){
        what++; //PROBLEM 1 = Counts how many times the target is found
        if (head->data == dataFind){
            howMany++;
            //PROBLEM 2 = Identify # of comparisons to find 18
            if (!countAll){
                return howMany;
            }
        }
        head = head->next; //MOVEMENT
    }
    return howMany;
}
#endif
```

node.h

OUTPUT

```
==============================
PROBLEM 1: SEQUENTIAL SEARCH
Searching for key 18
Array comparisons: 2
Linked list comparisons: 2
Answer: 2 comparisons are needed to find the first
==============================
PROBLEM 2: COUNT OCCURRENCES
Array occurrences of 18: 2
Linked list occurrences of 18: 2
Answer: 18 occurs 2 times.
==============================
PROBLEM 3: ITERATIVE BINARY SEARCH
Searching for key 8
Iteration #1
Low = 0
Middle = 4
Up = 9
Value = 11
Iteration #2
Low = 0
Middle = 1
Up = 3
Value = 5
Iteration #3
Low = 2
Middle = 2
Up = 3
Value = 6
Iteration #4
Low = 3
Middle = 3
Up = 3
Value = 8
Search is successful!
```

```
============================
PROBLEM 4: RECURSIVE BINARY SEARCH
Searching for key 8
Iteration #1
Low = 0
Middle = 4
Up = 9
Value = 11
Iteration #2
Low = 0
Middle = 1
Up = 3
Value = 5
Iteration #3
Low = 2
Middle = 2
Up = 3
Value = 6
Iteration #4
Low = 3
Middle = 3
Up = 3
Value = 8
Search was successful!
============================
```

Observation

The code is designed to test search algorithms on arrays and linked lists. It uses node.h to define the list nodes and build the memory connections, while searchAlgo.h holds the actual logic for linear and binary searches. Inside main.cpp, it builds a custom linked list alongside a simple array to run tests with values like 18 and 8. The program then prints out useful details, such as how many comparisons or iterations it took to find a match either iteratively or recursively. Overall, it gives a clear side-by-side look at how different searching algorithms step through structured data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.